

Encore, AWS & Agentic Development SDLC

A practical learning roadmap built around your existing TypeScript URL shortener

Target outcome

Deploy and evolve an Encore application using Encore's integrated platform services, with the application infrastructure provisioned into your own AWS account. Then introduce agentic engineering practices into the software delivery lifecycle.

Project approach: Encore-first. No Terraform requirement.

1. Objectives and working principles

This roadmap starts from the completed Encore TypeScript REST API tutorial and extends that application incrementally. The aim is to understand the platform while building a useful application—not to introduce a separate infrastructure stack.

Guiding principles

- Keep the existing TypeScript application as the starting point.
- Use Encore's native abstractions and integrated services wherever they fit.
- Deploy into an AWS account that you own, with Encore Cloud coordinating the deployment and provisioning supported infrastructure.
- Avoid introducing Terraform or a separate infrastructure-as-code workflow for this learning path.
- Add complexity in small, testable increments.
- Introduce agentic tooling only after the application and delivery workflow are understood.

Important distinction: Encore Cloud is the platform/control plane used to coordinate development workflows, deployments and infrastructure provisioning. The application workload and provisioned AWS resources run in your AWS account. Encore Cloud itself is not simply an EC2 instance running inside your account. Exact control-plane and data-plane boundaries, AWS resource types and account permissions should be confirmed against the current Encore documentation during setup.

2. Target architecture

Conceptually, the deployment has two sides:

Encore platform side

Encore Cloud provides the platform services that coordinate application environments, builds, deployments and infrastructure provisioning.

Your AWS account side

The deployed application and its provisioned runtime dependencies are hosted in your AWS account. The precise AWS services depend on the Encore features used and the deployment configuration.

A simplified view:

```
Developer workstation → Git repository → Encore Cloud deployment workflow → your AWS account → running Encore application and managed dependencies
```

Do not assume a fixed mapping such as “Encore always creates ECS Fargate and RDS”. Validate the actual resources created for the selected deployment and service configuration in the AWS console and Encore dashboard.

3. Learning phases

Phase 1 — Establish the baseline

- Keep the completed URL-shortener tutorial as the baseline application.
- Understand the service and API structure, request flow, database access and tests.
- Run the application locally and confirm the tests pass.
- Commit the working baseline to Git.

Phase 2 — Connect Encore Cloud and AWS

- Create or select an Encore application in the Encore platform.
- Link the local project to that application using the current CLI workflow.
- Connect the AWS account using Encore's documented account integration and least-privilege permissions.
- Choose a dedicated development environment and an appropriate AWS region.
- Review the deployment plan, required permissions, expected costs and resources before approving

provisioning.

Phase 3 — First AWS deployment

- Deploy the unchanged URL-shortener application.
- Find the deployed environment and public API endpoint.
- Exercise the API from outside your local machine.
- Inspect the AWS resources created and identify which are application compute, networking, database and supporting services.
- Compare the Encore dashboard view with the AWS console; record what Encore manages and what remains configurable by you.

Phase 4 — Expand the application

- Add one feature at a time, keeping changes small and covered by tests.
- Explore Encore-native services where appropriate: SQL databases, Pub/Sub, object storage, secrets, caching and scheduled jobs.
- For each service, understand its local development behaviour, cloud provisioning, configuration, permissions, observability and failure modes.
- Add operational features such as structured logging, request tracing, health checks, validation, rate limiting and error handling as appropriate.

Phase 5 — Introduce agentic application capabilities

- Define a narrow use case for an agent rather than adding an agent merely for novelty.
- Start with a bounded workflow, such as summarising URL metadata or classifying submitted links.
- Treat model calls and external tools as explicit dependencies; manage credentials as secrets.
- Define tool permissions, input validation, timeouts, retries, budgets and audit logging.
- Keep deterministic business rules outside the agent wherever possible, and require human approval for consequential actions.

Phase 6 — Build an agentic development SDLC

- Use coding agents for bounded tasks: explain code, propose changes, write tests and prepare pull requests.
- Require human review of plans, diffs, dependencies, permissions and generated infrastructure changes.
- Run formatting, linting, unit tests, integration tests and security checks in CI.
- Use branch protection and explicit approval gates before production deployment.
- Add evaluation datasets and regression tests for agent behaviour; track quality, latency, token/cost use and tool-call outcomes.
- Keep credentials scoped, avoid exposing production secrets to development agents, and log agent actions.

4. What to inspect during the AWS deployment

The first deployment is also an infrastructure-learning exercise. After Encore provisions the environment, inspect the following rather than assuming the architecture in advance:

Area	Questions to answer
Compute	Which AWS service runs the application? What is the deployment unit? How are tasks/instances scaled and replaced?
Networking	Which VPC, subnets, routes, security groups and load-balancing components exist? Which endpoints are public or private?
Database	Which database service is used? Where are credentials stored? What are backup, retention, encryption and connectivity settings?
Identity and access	Which IAM roles and policies are created? Which principal deploys? What permissions does the workload receive?
Observability	Where do logs, metrics, traces and deployment events appear? How do you correlate a request with its logs?
Cost and lifecycle	Which resources incur ongoing charges? How are environments deleted, and which data/resources persist?

5. Guardrails for the learning environment

- Use a dedicated AWS account or isolated development environment where practical.
- Select a single region and keep the initial deployment small.
- Review any estimated or actual AWS charges before leaving resources running.
- Do not put AWS access keys or model-provider keys in source control.
- Use least-privilege access and separate development from production credentials.
- Understand what deleting an Encore environment removes and what data may remain before testing teardown.

6. Suggested milestones and evidence of progress

Milestone	Evidence
Local baseline	Tutorial app runs; tests pass; code is committed.
Cloud connected	Encore application linked; AWS integration configured; region/environment selected.
First deployment	Public endpoint works; deployment status is healthy.
Infrastructure understood	Resource inventory documented; compute, networking, database and IAM roles identified.
Application expanded	At least one additional Encore service integrated and tested.
Agent feature	A bounded agent workflow works with defined permissions and evaluation tests.
Agentic SDLC	Coding-agent workflow, CI checks, human approvals and agent evaluations are documented.

7. Documentation to use

Use the current official documentation as the authority for CLI syntax, AWS account linking, supported regions, deployment architecture and platform behaviour. These details can change between Encore releases.

- [TypeScript REST API tutorial](#)
- [Encore Platform documentation](#)
- [Encore deployment documentation](#)
- [Encore self-hosting documentation](#) (for comparison later)

Scope note

This document is a learning plan, not a definitive AWS architecture or cost estimate. Encore's exact AWS resource topology and the division of responsibilities between Encore Cloud and the AWS account must be confirmed during the live account-connection and deployment workflow. Do not approve provisioning until the proposed resources, permissions and costs are understood.